SANJIVANI UNIVERSITY

Department of Computer Science and Engineering

IM.Tech Programme

LABORATORY MANUAL

EXPLORATORY DATA ANALYSIS

Course Code: 25PIMPC304

Academic Year: 2026–27

Prepared By

VIKAS KUMAR

Assistant Professor

Department of Computer Science and Engineering

2026–27

Contents

1 Automated Data Profiling and Sanity Checks **6**
1.1 Experiment Number . . . 6
1.2 Title . . . 6
1.3 Relevant CO . . . 6
1.4 Aim . . . 6
1.5 Objective . . . 6
1.6 Theory . . . 6
1.7 Software Requirements . . . 7
1.8 Dataset . . . 7
1.9 Procedure . . . 7
1.10 Python Implementation . . . 8
1.11 Automated Profiling . . . 8
1.12 Expected Observations . . . 8
1.13 Deliverable . . . 9
1.14 Result . . . 9
1.15 Viva Questions . . . 9

2 Validating Data Quality with Code Assertions **10**
2.1 Experiment Number . . . 10
2.2 Title . . . 10
2.3 Relevant CO . . . 10
2.4 Aim . . . 10
2.5 Objective . . . 10
2.6 Theory . . . 10
2.7 Procedure . . . 11
2.8 Python Implementation . . . 11
2.9 Unique Key Validation . . . 11
2.10 Age Validation . . . 11
2.11 Income Validation . . . 11
2.12 Date Validation . . . 12
2.13 Email Validation . . . 12
2.14 Dirty Row Percentage . . . 12
2.15 Deliverable . . . 13
2.16 Result . . . 13
2.17 Viva Questions . . . 13

3 Algorithmic Imputation and Outlier Treatment **14**
3.1 Experiment Number . . . 14
3.2 Title . . . 14
3.3 Relevant CO . . . 14
3.4 Aim . . . 14
3.5 Objective . . . 14
3.6 Theory . . . 14
3.7 Mean Imputation . . . 15
3.8 KNN Imputation . . . 15
3.9 IQR Outlier Detection . . . 15
3.10 Z-score Method . . . 16

3.11 RobustScaler . . . 16
3.12 Deliverable . . . 16
3.13 Result . . . 16
3.14 Viva Questions . . . 16

4 Feature Engineering and the Tidy Data Pipeline 18
4.1 Experiment Number . . . 18
4.2 Title . . . 18
4.3 Relevant CO . . . 18
4.4 Aim . . . 18
4.5 Objective . . . 18
4.6 Theory . . . 18
4.7 Melt . . . 18
4.8 Pivot Table . . . 19
4.9 Datetime Feature Engineering . . . 19
4.10 Target Encoding . . . 19
4.11 Pipeline . . . 19
4.12 Deliverable . . . 20
4.13 Result . . . 20
4.14 Viva Questions . . . 20

5 Statistical Distribution and Normality Testing 21
5.1 Experiment Number . . . 21
5.2 Title . . . 21
5.3 Relevant CO . . . 21
5.4 Aim . . . 21
5.5 Objective . . . 21
5.6 Theory . . . 21
5.7 Histogram and KDE . . . 22
5.8 Skewness . . . 22
5.9 Kurtosis . . . 22
5.10 Shapiro-Wilk Test . . . 22
5.11 Q-Q Plot . . . 23
5.12 Deliverable . . . 23
5.13 Result . . . 23
5.14 Viva Questions . . . 23

6 Bivariate Relationship Mapping and Correlation Analysis 24
6.1 Experiment Number . . . 24
6.2 Title . . . 24
6.3 Relevant CO . . . 24
6.4 Aim . . . 24
6.5 Objective . . . 24
6.6 Theory . . . 24
6.7 Cross-tabulation . . . 24
6.8 Stacked Bar Chart . . . 25
6.9 Scatterplot . . . 25
6.10 LOESS Trend . . . 25
6.11 Pearson Correlation . . . 25

6.12 Spearman Correlation . . . 25
6.13 Correlation Heatmap . . . 26
6.14 Deliverable . . . 26
6.15 Result . . . 26
6.16 Viva Questions . . . 26

7 Multi-Dimensional Visualization and Simpson's Paradox 27
7.1 Experiment Number . . . 27
7.2 Title . . . 27
7.3 Relevant CO . . . 27
7.4 Aim . . . 27
7.5 Objective . . . 27
7.6 Theory . . . 27
7.7 FacetGrid . . . 28
7.8 Parallel Coordinates . . . 28
7.9 Overall Success Rate . . . 28
7.10 Grouped Success Rate . . . 28
7.11 Deliverable . . . 29
7.12 Result . . . 29
7.13 Viva Questions . . . 29

8 Dimensionality Reduction for High-Dimensional Data 30
8.1 Experiment Number . . . 30
8.2 Title . . . 30
8.3 Relevant CO . . . 30
8.4 Aim . . . 30
8.5 Objective . . . 30
8.6 Theory . . . 30
8.7 PCA . . . 30
8.8 Scree Plot . . . 31
8.9 t-SNE . . . 31
8.10 UMAP . . . 32
8.11 Deliverable . . . 32
8.12 Result . . . 32
8.13 Viva Questions . . . 32

9 Designing Interactive Visualizations with Plotly 33
9.1 Experiment Number . . . 33
9.2 Title . . . 33
9.3 Relevant CO . . . 33
9.4 Aim . . . 33
9.5 Objective . . . 33
9.6 Theory . . . 33
9.7 Python Implementation . . . 34
9.8 Hover Information . . . 34
9.9 Range Slider . . . 34
9.10 Save HTML . . . 34
9.11 Deliverable . . . 34
9.12 Result . . . 35

9.13 Viva Questions 35

10 Building and Deploying an EDA Web Dashboard 36
10.1 Experiment Number 36
10.2 Title 36
10.3 Relevant CO 36
10.4 Aim 36
10.5 Objective 36
10.6 Theory 36
10.7 Software Requirements 36
10.8 Create Application 37
10.9 Python Implementation 37
10.10Interactive Sidebar 37
10.11Interactive Plot 38
10.12Run the Application 38
10.13Deliverable 38
10.14Result 38
10.15Viva Questions 38

Course Information

Course Name	Exploratory Data Analysis
Course Code	25PIMPC304
Programme	iM.Tech
Department	Computer Science and Engineering
Academic Year	2026–27

Laboratory Objectives

The laboratory is designed to provide practical knowledge of Exploratory Data Analysis and modern data analytics techniques.

Students will learn to:

1. Inspect and profile real-world datasets.
2. Identify and validate data-quality problems.
3. Handle missing values and outliers.
4. Perform feature engineering and data transformation.
5. Analyze statistical distributions.
6. Study relationships between variables.
7. Understand multivariate relationships and confounding.
8. Apply dimensionality reduction techniques.
9. Create interactive visualizations.
10. Develop an interactive EDA dashboard.

1 Automated Data Profiling and Sanity Checks

1.1 Experiment Number

1

1.2 Title

Automated Data Profiling and Sanity Checks

1.3 Relevant CO

CO PC304.1

1.4 Aim

To initialize an Exploratory Data Analysis pipeline and rapidly assess the structure, quality and statistical characteristics of a raw dataset using Pandas and automated profiling tools.

1.5 Objective

The objectives of this experiment are:

1. To load and inspect a real-world dataset.
2. To identify data types and structural characteristics.
3. To detect missing values and duplicate observations.
4. To generate an automated profiling report.
5. To identify important data-quality warnings.

1.6 Theory

Exploratory Data Analysis (EDA) is the process of examining, summarizing and visualizing a dataset before applying statistical analysis or machine learning techniques.

Real-world datasets may contain:

- Missing values
- Duplicate observations
- Incorrect data types
- Outliers
- Skewed variables
- Highly correlated variables

Initial data profiling helps identify these problems before performing further analysis. Pandas provides several functions for initial inspection:

```
df.head()
df.info()
df.describe()
df.shape
df.isnull().sum()
df.duplicated().sum()
```

Automated profiling tools such as YData Profiling generate reports containing variable information, missing values, distributions, correlations and duplicate observations.

1.7 Software Requirements

- Python 3.x
- Jupyter Notebook or Google Colab
- Pandas
- NumPy
- YData Profiling

1.8 Dataset

A real-world dataset such as retail, healthcare, financial, customer transaction or sales data may be used.

1.9 Procedure

1. Import the required libraries.
2. Load the dataset using Pandas.
3. Display the first few records.
4. Check the dimensions of the dataset.
5. Examine the data types.
6. Generate descriptive statistics.
7. Check missing values.
8. Check duplicate observations.
9. Generate an automated profiling report.
10. Identify at least three important warnings.

1.10 Python Implementation

```
import pandas as pd
import numpy as np

df = pd.read_csv("retail_data.csv")

print(df.head())

print("Shape:", df.shape)

df.info()

print(df.describe())

print("Missing values:")
print(df.isnull().sum())

print("Duplicate rows:")
print(df.duplicated().sum())
```

1.11 Automated Profiling

```
from ydata_profiling import ProfileReport

profile = ProfileReport(
    df,
    title="EDA Data Profiling Report"
)

profile.to_file(
    "profiling_report.html"
)
```

1.12 Expected Observations

Students should document:

- Number of rows and columns
- Numerical variables
- Categorical variables
- Missing-value percentage
- Number of duplicate records
- Highly correlated variables
- Highly skewed variables

1.13 Deliverable

Prepare a short "Data Quality Audit Report".

1.14 Result

The dataset was successfully loaded, inspected and profiled. Important data-quality issues were identified before further analysis.

1.15 Viva Questions

1. What is Exploratory Data Analysis?
2. Why is data profiling important?
3. What is the difference between `info()` and `describe()`?
4. What are missing values?
5. What is automated data profiling?

2 Validating Data Quality with Code Assertions

2.1 Experiment Number

2

2.2 Title

Validating Data Quality with Code Assertions

2.3 Relevant CO

CO PC304.1

2.4 Aim

To programmatically define and test data-quality rules using Python assertions and identify logically invalid records.

2.5 Objective

1. To identify duplicate rows.
2. To detect unique-key violations.
3. To validate numerical ranges.
4. To validate date values.
5. To identify invalid email formats.
6. To calculate the percentage of dirty records.

2.6 Theory

Data validation is the process of checking whether values in a dataset satisfy predefined constraints.

For example:

$$Age \geq 0$$

$$Age \leq 120$$

$$Income \geq 0$$

Assertions allow these rules to be checked automatically.

2.7 Procedure

1. Load the dataset.
2. Identify duplicate rows.
3. Check duplicate customer IDs.
4. Validate age values.
5. Validate income values.
6. Validate date values.
7. Validate email formats.
8. Calculate the percentage of dirty rows.

2.8 Python Implementation

```
duplicates = df[df.duplicated()]

print(duplicates)

print(
    "Number of duplicates:",
    len(duplicates)
)
```

2.9 Unique Key Validation

```
duplicate_ids = df[
    df["Customer_ID"].duplicated()
]

print(duplicate_ids)
```

2.10 Age Validation

```
assert df["Age"].min() >= 0

assert df["Age"].max() <= 120
```

2.11 Income Validation

```
assert df["Income"].min() >= 0
```

2.12 Date Validation

```
df["Registration_Date"] = pd.to_datetime(
    df["Registration_Date"]
)

today = pd.Timestamp.today()

invalid_dates = df[
    df["Registration_Date"] > today
]

print(invalid_dates)
```

2.13 Email Validation

```
email_pattern = r"^[\w\.-]+@[\w\.-]+\.\w+$"

invalid_email = df[
    ~df["Email"].str.match(
        email_pattern,
        na=False
    )
]

print(invalid_email)
```

2.14 Dirty Row Percentage

```
invalid_age = (
    (df["Age"] < 0) |
    (df["Age"] > 120)
)

invalid_income = df["Income"] < 0

dirty = invalid_age | invalid_income

percentage_dirty = dirty.mean() * 100

print("Dirty rows:", dirty.sum())

print(
    "Percentage:",
    percentage_dirty
)
```

2.15 Deliverable

Submit a Python script containing robust sanity checks and an output log of violations.

2.16 Result

The data-quality rules were successfully implemented and invalid records were identified programmatically.

2.17 Viva Questions

1. What is data validation?
2. What is an assertion?
3. What is a unique-key violation?
4. Why is validation required before machine learning?
5. What is a dirty record?

3 Algorithmic Imputation and Outlier Treatment

3.1 Experiment Number

3

3.2 Title

Algorithmic Imputation and Outlier Treatment

3.3 Relevant CO

CO PC304.2

3.4 Aim

To compare mean imputation with KNN imputation and detect and treat outliers using statistical methods.

3.5 Objective

1. To handle missing values using mean imputation.
2. To apply KNN imputation.
3. To detect outliers using IQR.
4. To detect outliers using Z-score.
5. To apply RobustScaler.
6. To compare distributions before and after treatment.

3.6 Theory

Missing values are common in real-world datasets.

Mean imputation replaces a missing value by the mean:

$$x_{missing} = \bar{x}$$

KNN imputation estimates missing values using similar observations.

The Interquartile Range is:

$$IQR = Q_3 - Q_1$$

The lower boundary is:

$$Q_1 - 1.5(IQR)$$

The upper boundary is:

$$Q_3 + 1.5(IQR)$$

The Z-score is:

$$Z = \frac{x - \mu}{\sigma}$$

3.7 Mean Imputation

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(
    strategy="mean"
)

df["Income_Mean"] = imputer.fit_transform(
    df[["Income"]]
)
```

3.8 KNN Imputation

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(
    n_neighbors=5
)

columns = [
    "Income",
    "Age",
    "LoanAmount"
]

df[columns] = imputer.fit_transform(
    df[columns]
)
```

3.9 IQR Outlier Detection

```
Q1 = df["Income"].quantile(0.25)

Q3 = df["Income"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR

upper = Q3 + 1.5 * IQR

outliers = df[
    (df["Income"] < lower) |
```

```
    (df["Income"] > upper)
]

print(outliers)
```

3.10 Z-score Method

```
from scipy.stats import zscore

df["Income_Z"] = zscore(
    df["Income"]
)

outliers = df[
    abs(df["Income_Z"]) > 3
]

print(outliers)
```

3.11 RobustScaler

Robust scaling uses the median and IQR:

$$x' = \frac{x - \text{Median}(x)}{Q_3 - Q_1}$$

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()

df["Income_Robust"] = scaler.fit_transform(
    df[["Income"]]
)
```

3.12 Deliverable

Submit before-and-after histograms and a comparison of mean imputation and KNN imputation.

3.13 Result

Missing values and outliers were successfully analyzed and treated using statistical and algorithmic methods.

3.14 Viva Questions

1. What is missing-value imputation?
2. Why can mean imputation cause distortion?

3. What is KNN imputation?
4. What is IQR?
5. What is RobustScaler?

4 Feature Engineering and the Tidy Data Pipeline

4.1 Experiment Number

4

4.2 Title

Feature Engineering and the "Tidy Data" Pipeline

4.3 Relevant CO

CO PC304.2

4.4 Aim

To transform messy datasets into analytical form using reshaping, datetime feature extraction and categorical encoding.

4.5 Objective

1. To reshape data using `melt()`.
2. To create summary tables using `pivot_table()`.
3. To extract datetime features.
4. To encode high-cardinality categorical variables.
5. To create a reproducible preprocessing pipeline.

4.6 Theory

Tidy data follows three basic principles:

1. Each variable forms a column.
2. Each observation forms a row.
3. Each observational unit forms a table.

4.7 Melt

```
tidy_df = pd.melt(
    df,
    id_vars=["Customer_ID"],
    var_name="Variable",
    value_name="Value"
)

print(tidy_df.head())
```

4.8 Pivot Table

```
pivot = df.pivot_table(
    values="Sales",
    index="City",
    columns="Month",
    aggfunc="sum"
)

print(pivot)
```

4.9 Datetime Feature Engineering

```
df["Date"] = pd.to_datetime(
    df["Date"]
)

df["Day"] = df["Date"].dt.day

df["Month"] = df["Date"].dt.month

df["Is_Weekend"] = (
    df["Date"].dt.dayofweek >= 5
)
```

4.10 Target Encoding

For category c:

$$TE(c) = \frac{\sum y_i}{N_c}$$

```
import category_encoders as ce

encoder = ce.TargetEncoder(
    cols=["City"]
)

df["City_encoded"] = encoder.fit_transform(
    df["City"],
    df["Target"]
)
```

4.11 Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import RobustScaler
```

```
pipeline = Pipeline([
    ("imputer",
     SimpleImputer(strategy="median")),
    ("scaler",
     RobustScaler())
])
```

4.12 Deliverable

Submit a reproducible preprocessing pipeline.

4.13 Result

The messy dataset was transformed into a structured analytical dataset using feature engineering and preprocessing techniques.

4.14 Viva Questions

1. What is tidy data?
2. What is the purpose of `melt()`?
3. What is a pivot table?
4. What is feature engineering?
5. What is target encoding?

5 Statistical Distribution and Normality Testing

5.1 Experiment Number

5

5.2 Title

Statistical Distribution and Normality Testing

5.3 Relevant CO

CO PC304.3

5.4 Aim

To evaluate the shape, central tendency, dispersion and normality of continuous variables.

5.5 Objective

1. To plot histograms with KDE.
2. To calculate skewness.
3. To calculate kurtosis.
4. To perform the Shapiro-Wilk test.
5. To generate Q-Q plots.

5.6 Theory

A probability distribution describes how values of a variable are distributed.
Important characteristics include:

- Mean
- Median
- Variance
- Standard deviation
- Skewness
- Kurtosis

5.7 Histogram and KDE

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(
    df["Height"],
    kde=True
)

plt.show()
```

5.8 Skewness

```
from scipy.stats import skew

value = skew(
    df["Height"].dropna()
)

print("Skewness:", value)
```

Positive skewness indicates right skewness, while negative skewness indicates left skewness.

5.9 Kurtosis

```
from scipy.stats import kurtosis

k = kurtosis(
    df["Height"].dropna()
)

print("Kurtosis:", k)
```

5.10 Shapiro-Wilk Test

The hypotheses are:

$$H_0 : \text{Data follows a normal distribution}$$

$$H_1 : \text{Data does not follow a normal distribution}$$

```
from scipy.stats import shapiro

stat, p = shapiro(
    df["Height"].dropna()
)
```

```
print("Statistic:", stat)

print("p-value:", p)
```

If $p < 0.05$, the null hypothesis is rejected.
If $p > 0.05$, the null hypothesis is not rejected.

5.11 Q-Q Plot

```
from scipy.stats import probplot

probplot(
    df["Height"].dropna(),
    dist="norm",
    plot=plt
)

plt.show()
```

5.12 Deliverable

Provide a statistical conclusion regarding the normality of the selected variables.

5.13 Result

The distribution and normality of continuous variables were evaluated using graphical and statistical techniques.

5.14 Viva Questions

1. What is skewness?
2. What is kurtosis?
3. What is a normal distribution?
4. What is the null hypothesis of the Shapiro-Wilk test?
5. What is a Q-Q plot?

6 Bivariate Relationship Mapping and Correlation Analysis

6.1 Experiment Number

6

6.2 Title

Bivariate Relationship Mapping and Correlation Analysis

6.3 Relevant CO

CO PC304.3

6.4 Aim

To identify and quantify relationships between continuous and categorical variables.

6.5 Objective

1. To create cross-tabulations.
2. To create stacked bar charts.
3. To generate scatterplots.
4. To visualize LOESS trends.
5. To calculate Pearson correlation.
6. To calculate Spearman correlation.

6.6 Theory

Correlation measures the strength and direction of association between variables.

Pearson correlation is:

$$r = \frac{Cov(X, Y)}{\sigma_X \sigma_Y}$$

Pearson correlation mainly measures linear relationships.

Spearman correlation is based on ranks and is useful for monotonic relationships.

6.7 Cross-tabulation

```
table = pd.crosstab(
    df["Property_Type"],
    df["Location"]
)

print(table)
```

6.8 Stacked Bar Chart

```
table.plot(
    kind="bar",
    stacked=True
)

plt.show()
```

6.9 Scatterplot

```
sns.scatterplot(
    data=df,
    x="Area",
    y="Price"
)

plt.show()
```

6.10 LOESS Trend

```
sns.regplot(
    data=df,
    x="Area",
    y="Price",
    lowess=True
)

plt.show()
```

6.11 Pearson Correlation

```
pearson = df.corr(
    numeric_only=True,
    method="pearson"
)

print(pearson)
```

6.12 Spearman Correlation

```
spearman = df.corr(
    numeric_only=True,
    method="spearman"
)

print(spearman)
```

6.13 Correlation Heatmap

```
sns.heatmap(
    pearson,
    annot=True,
    cmap="coolwarm"
)

plt.title(
    "Pearson Correlation"
)

plt.show()
```

6.14 Deliverable

Identify the three strongest relationships with the target variable.

6.15 Result

Important bivariate relationships were identified using visualization and correlation analysis.

6.16 Viva Questions

1. What is Pearson correlation?
2. What is Spearman correlation?
3. What is LOESS?
4. Does correlation imply causation?
5. How can outliers affect correlation?

7 Multi-Dimensional Visualization and Simpson's Paradox

7.1 Experiment Number

7

7.2 Title

Multi-Dimensional Visualization and Simpson's Paradox

7.3 Relevant CO

CO PC304.4

7.4 Aim

To visualize interactions among three or more variables and identify confounding factors using Simpson's Paradox.

7.5 Objective

1. To visualize multidimensional data.
2. To calculate overall success rates.
3. To calculate group-wise success rates.
4. To identify a confounding variable.
5. To explain Simpson's Paradox.

7.6 Theory

Simpson's Paradox occurs when a relationship observed in aggregated data changes or reverses after the data is divided according to a third variable.

For example:

$$P(Success|A) > P(Success|B)$$

may become:

$$P(Success|A, D) < P(Success|B, D)$$

after considering a third variable D.

7.7 FacetGrid

```
g = sns.FacetGrid(
    df,
    col="Department",
    hue="Gender"
)

g.map(
    sns.scatterplot,
    "Score",
    "Success"
)

g.add_legend()
```

7.8 Parallel Coordinates

```
from pandas.plotting import parallel_coordinates

parallel_coordinates(
    df[
        ["Department",
         "Score",
         "Income",
         "Success"]
    ],
    "Department"
)

plt.show()
```

7.9 Overall Success Rate

```
overall = df["Success"].mean()

print(
    "Overall success:",
    overall
)
```

7.10 Grouped Success Rate

```
grouped = df.groupby(
    "Department"
)["Success"].mean()

print(grouped)
```

7.11 Deliverable

Prepare a visual storyboard containing:

1. Aggregate relationship
2. Group-wise relationship
3. Confounding variable
4. Explanation of the reversal

7.12 Result

The experiment demonstrates how aggregate statistics can be misleading and how multivariate grouping can reveal the actual relationship.

7.13 Viva Questions

1. What is Simpson's Paradox?
2. What is a confounding variable?
3. Why can aggregation be misleading?
4. What is multivariate analysis?

8 Dimensionality Reduction for High-Dimensional Data

8.1 Experiment Number

8

8.2 Title

Dimensionality Reduction for High-Dimensional Data

8.3 Relevant CO

CO PC304.4

8.4 Aim

To reduce high-dimensional feature spaces while retaining important information using PCA, t-SNE and UMAP.

8.5 Objective

1. To calculate Variance Inflation Factor.
2. To detect multicollinearity.
3. To apply PCA.
4. To generate a scree plot.
5. To apply t-SNE or UMAP.
6. To visualize high-dimensional data in two dimensions.

8.6 Theory

Dimensionality reduction transforms a high-dimensional dataset into a lower-dimensional representation while attempting to retain important information.

Variance Inflation Factor is:

$$VIF_j = \frac{1}{1 - R_j^2}$$

8.7 PCA

Principal Component Analysis transforms correlated variables into orthogonal principal components.

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X_scaled = StandardScaler().fit_transform(X)
```

```
pca = PCA()

X_pca = pca.fit_transform(
    X_scaled
)

print(
    pca.explained_variance_ratio_
)
```

8.8 Scree Plot

```
plt.plot(
    range(
        1,
        len(
            pca.explained_variance_ratio_
        ) + 1
    ),
    pca.explained_variance_ratio_,
    marker="o"
)

plt.xlabel(
    "Principal Component"
)

plt.ylabel(
    "Explained Variance"
)

plt.title(
    "Scree Plot"
)

plt.show()
```

8.9 t-SNE

```
from sklearn.manifold import TSNE

tsne = TSNE(
    n_components=2,
    random_state=42
)

X_tsne = tsne.fit_transform(
    X_scaled
)
```

8.10 UMAP

```
import umap

reducer = umap.UMAP(
    n_components=2,
    random_state=42
)

X_umap = reducer.fit_transform(
    X_scaled
)
```

8.11 Deliverable

Submit a 2D visualization and explain the cluster structure obtained after dimensionality reduction.

8.12 Result

The high-dimensional dataset was transformed into a lower- dimensional representation for visualization and analysis.

8.13 Viva Questions

1. What is dimensionality reduction?
2. What is PCA?
3. What is explained variance?
4. What is VIF?
5. Compare PCA, t-SNE and UMAP.

9 Designing Interactive Visualizations with Plotly

9.1 Experiment Number

9

9.2 Title

Designing Interactive Visualizations with Plotly

9.3 Relevant CO

CO PC304.5

9.4 Aim

To convert static charts into interactive visualizations using Plotly.

9.5 Objective

1. To create interactive charts.
2. To add hover information.
3. To implement zooming and panning.
4. To implement a range slider.
5. To save visualizations as HTML files.

9.6 Theory

Interactive visualizations allow users to explore data dynamically rather than viewing a static graph.

Plotly provides features such as:

- Hover information
- Zooming
- Panning
- Interactive legends
- Range sliders

9.7 Python Implementation

```
import plotly.express as px

df["Date"] = pd.to_datetime(
    df["Date"]
)

fig = px.line(
    df,
    x="Date",
    y="Price",
    title="Time-Series Analysis"
)

fig.show()
```

9.8 Hover Information

```
fig = px.line(
    df,
    x="Date",
    y="Price",
    hover_data=["Volume"]
)

fig.show()
```

9.9 Range Slider

```
fig.update_xaxes(
    rangeslider_visible=True
)

fig.show()
```

9.10 Save HTML

```
fig.write_html(
    "interactive_eda.html"
)
```

9.11 Deliverable

Submit an interactive HTML file containing a zoomable and interactive visualization.

9.12 Result

An interactive visualization was successfully created using Plotly.

9.13 Viva Questions

1. What is interactive visualization?
2. What is Plotly?
3. What is a range slider?
4. What is hover information?
5. Why is interactivity useful in EDA?

10 Building and Deploying an EDA Web Dashboard

10.1 Experiment Number

10

10.2 Title

Building and Deploying an EDA Web Dashboard

10.3 Relevant CO

CO PC304.5

10.4 Aim

To develop an interactive web-based EDA application using Streamlit and Plotly.

10.5 Objective

1. To create a Streamlit application.
2. To upload a dataset.
3. To display dataset information.
4. To create interactive sidebar controls.
5. To dynamically filter the dataset.
6. To generate interactive Plotly visualizations.

10.6 Theory

Streamlit is a Python framework for developing interactive data applications.

The general workflow is:

$$Dataset \rightarrow Processing \rightarrow Filtering \rightarrow Visualization \rightarrow Dashboard$$

10.7 Software Requirements

- Python 3.x
- Streamlit
- Pandas
- Plotly

10.8 Create Application

Create a Python file named:

```
app.py
```

10.9 Python Implementation

```
import streamlit as st
import pandas as pd
import plotly.express as px

st.set_page_config(
    page_title="EDA Dashboard",
    layout="wide"
)

st.title(
    "Exploratory Data Analysis Dashboard"
)

uploaded_file = st.file_uploader(
    "Upload CSV file",
    type=["csv"]
)

if uploaded_file is not None:

    df = pd.read_csv(
        uploaded_file
    )

    st.subheader(
        "Dataset Information"
    )

    st.write(
        "Rows:",
        df.shape[0]
    )

    st.write(
        "Columns:",
        df.shape[1]
    )

    st.dataframe(df)
```

10.10 Interactive Sidebar

```
numeric_columns = df.select_dtypes(
    include="number"
).columns

x = st.sidebar.selectbox(
    "X Variable",
    numeric_columns
)

y = st.sidebar.selectbox(
    "Y Variable",
    numeric_columns
)
```

10.11 Interactive Plot

```
fig = px.scatter(
    df,
    x=x,
    y=y,
    title="Interactive Relationship"
)

st.plotly_chart(
    fig,
    use_container_width=True
)
```

10.12 Run the Application

Use the following command in the terminal:

```
streamlit run app.py
```

10.13 Deliverable

Submit a functioning Streamlit-based EDA dashboard.

10.14 Result

A functional interactive EDA dashboard was developed using Streamlit and Plotly.

10.15 Viva Questions

1. What is Streamlit?
2. What are Streamlit widgets?

3. How can Plotly be integrated with Streamlit?
4. What is an interactive dashboard?
5. How can an EDA dashboard be deployed?

Overall Learning Outcomes

After completing the laboratory, students will be able to:

1. Perform automated data profiling.
2. Validate data quality programmatically.
3. Handle missing values and outliers.
4. Perform feature engineering.
5. Analyze statistical distributions.
6. Perform bivariate and multivariate analysis.
7. Understand Simpson's Paradox.
8. Apply dimensionality reduction techniques.
9. Create interactive visualizations.
10. Develop an interactive EDA dashboard.

Experiment Summary

No.	Experiment	CO
1	Automated Data Profiling	PC304.1
2	Data Quality Validation	PC304.1
3	Imputation and Outlier Treatment	PC304.2
4	Feature Engineering	PC304.2
5	Statistical Distribution Analysis	PC304.3
6	Bivariate and Correlation Analysis	PC304.3
7	Multivariate Analysis	PC304.4
8	Dimensionality Reduction	PC304.4
9	Interactive Visualization	PC304.5
10	EDA Web Dashboard	PC304.5

Practical Record Format

For every experiment, students should include:

1. Experiment Number
2. Title
3. Aim
4. Objectives
5. Theory

6. Software Requirements
7. Dataset Description
8. Procedure
9. Python Implementation
10. Output
11. Observations
12. Result
13. Viva Questions

Assessment Scheme

Component	Marks
Theory and Understanding	10
Dataset Preparation	10
Python Implementation	25
Visualization	15
Statistical Interpretation	15
Documentation	10
Viva	15
Total	**100**